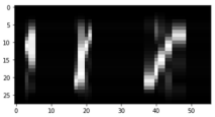
Master 2 **Machine Learning on Sequences**

Practical work 2:
Hidden Markov Models (HMM)
Forward-Backward versus Viterbi training

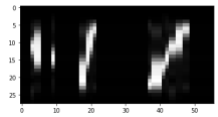
In this session we focus on training semi continuous Hidden Makov Models for handwritten digit recognition. We recall that images are 28X28 as shown on figure below.

Exemple of image digit of the MNIST dataset.

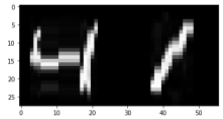
Recall that semi-continuous HMM are discreet HMM that have discret input sequences where the input have been discretized through a clustering preprocessing stage, which in the present case assigns a single ID to every columns of 28 pixels as depicted on figures below. The size of the codebook induces a discretization effect, the larger the codebook, the lower the discretization error. Notice that the codebook is shared by the different HMM. In this exercise you can choose to use two different codebooks of size 50 and 100 clusters.



0 0 0 5 1 1 0 0 0 0 0 0 0 0 0 0 0 0 4 7 7 5 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 4 4 4 6 9 9 5 5 2 2 2 2 0 0 0 0 0 0 0



1 1 1 19 2 2 1 1 1 8 1 1 1 1 1 1 1 5 5 2 6 16 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 12 5 5 5 4 8 19 19 6 6 16 16 1 1 1 1 1 1 1



2 2 2 21 8 5 0 0 0 0 0 21 21 21 21 21 21 6 4 33 13 11 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 39 6 6 36 7 32 40 40 13 1 11 11 2 2 2 2 2 2 2 2

Discretization effect using 10, 20, 50 clusters, and the corresponding to a discreet sequence.

1. Training HMM with the forward-backward algorithm

Training a HMM with the forward – backward algorithm takes into account every possible hidden state sequences to compute the statistics that are necessary for estimating the optimal parameters of a new HMM.

We recall the re-estimations formulas obtained:

$$\pi^*_i = \frac{\sum_{l=1}^{L} \frac{1}{p_l} \alpha_0^l(i)\beta_0^l(i)}{\sum_{l=1}^{L} \frac{1}{p_l} \sum_{k=0}^{K-1} \alpha_0^l(k)\beta_0^l(k)}$$

$$b_i^*(v_m) = \frac{\sum_{l=1}^{L} \frac{1}{p_l} \sum_{\substack{t=0 \\ o_{t+1}^l = v_m}}^{T-1} \alpha_t^l(i)\beta_t^l(i)}{\sum_{l=1}^{L} \frac{1}{p_l} \sum_{t=0}^{T-2} \alpha_t^l(i)\beta_t^l(i)}$$

$$a_{ij}^* = \frac{\sum_{l=1}^{L} \frac{1}{p_l} \sum_{t=0}^{T-2} \alpha_t^l(i) a_{ij} b_j(o_{t+1}^l)\, \beta_{t+1}^l(i)}{\sum_{l=1}^{L} \frac{1}{p_l} \sum_{t=0}^{T-2} \alpha_t^l(i)\beta_t^l(i)}$$

with

$$p_l = P(O^l) = \sum_{i=0}^{K-1} \alpha_T(i)\beta_T(i)$$

Observe that we can also take account of the finite duration of the observation sequences and state sequences as well, such that we can introduce final states probabilities Pf_i that are the probability for state s_i to end the sequence, $Pf_i = p(q_{T-1} = i)$.
The re-estimation formula for final probability is the following:

$$Pf_i^* = \frac{\sum_{l=1}^{L} \frac{1}{p_l} \alpha_{T-1}^l(i)\beta_{T-1}^l(i)}{\sum_{l=1}^{L} \frac{1}{p_l} \sum_{k=0}^{K-1} \alpha_{T-1}^l(k)\beta_{T-1}^l(k)}$$

Question 1: Training with multiple examples.

The re-estimation formulas below suggest that we need to sum the contribution of each training example l to the numerator and to the denominator of each of the tree quantities to estimate.

Observe that since we work with *left-to-right* models the initial probability and final probability vectors remain unchanged during training, and there is no need to res-estimate them.

Write the following methods that cumulate the statistics (in log scale) of the numerators and the denominator of the re-estimation formulas.

```
def cumulate_den(self,T,fwd,bwd, sum_den,log_likelihood):
```

```
def cumulate_A_num(self,train_data,fwd_lattice,bwd_lattice,sum_A_num,
log_likelihood):
```

```
def cumulate_B_num(self,train_data,fwd,bwd, sum_B_num,log_likelihood):
```

with the following variables:

$sum\_B\_num[i,v]$ $with\ i = 0,\dots,n\_states - 1$ and $v = 0,\dots,n\_clusters\ - 1$
$sum\_A\_num[i,j]$ $with\ i = 0,\dots,n\_states - 1$ and $j = 0,\dots,n\_states\ - 1$
$sum\_den[i]$ $with\ i = 0,\dots,n\_states - 1$

Such that the estimation step, the M step, corresponds to the following method:

```
def Mstep(self,sum_B_num, sum_den, sum_A_num):

        SUM_DEN_A =np.repeat(sum_den,self.n_states).reshape(self.n_states,
                    self.n_states)
        SUM_DEN_B = np.repeat(sum_den,self.n_clusters).reshape
                     (self.n_states,self.n_clusters)

        self.log_A = sum_A_num - SUM_DEN_A
        self.log_B = sum_B_num - SUM_DEN_B

        self.log_B[np.equal(self.log_B,float('-inf'))] = self.log_epsilon
```

Question 2: Digit recognition

- Use the given program to train the 10 digit models. At each iteration of the EM algorithm the model is tested on the validation dataset by calling the `Log_likelihood(valid_data)` method. The best model is saved through the iterations. The number of EM iterations is fixed to Max_EM_iter

- Comment and analyze the training and validation loss (images saved)

- Run the recognition step on the MNIST test dataset composed of 10 000 digit images by putting `TRAINING = False`

- Fill in the table below where RR stand for the Recognition Rate computed on un-seen examples during training, i.e. on the test set made of 10 000 examples.

- Draw some conclusion.

N_train	*n_cluster*	*n_states*	*RR (%) test*
10 000	*50*	*5*	-
	50	*10*	-
	50	*15*	-
	100	*5*	-
	100	*10*	-
10 000	*100*	*15*	-
54 000	*100*	*15*	-

2. Training HMM with the Viterbi algorithm

Question 3: Training with VITERBI

Using the Viterbi algorithm, it is possible de estimate the missing hidden states using a hard assignment.

- Write the methods that cumulates the statics along the Viterbi path.

```
def cumulate_den_viterbi(self,T,fwd,bwd, sum_den,log_likelihood):
```

```
def cumulate_A_num_viterbi(self,train_data,fwd_lattice,bwd_lattice,sum_A_num,
log_likelihood):
```

```
def cumulate_B_num_viterbi(self,train_data,fwd,bwd, sum_B_num,log_likelihood):
```

- Write the methods that trains a HMM using Viterbi, filling the method TrainViterbi defined below and provided in hmm.py.

```
def TrainViterbi(self,classe,train_data,valid_data,Max_EM_iter,dir_name,file_name):

return LL_train_history/n_train, LL_valid_history/n_valid, best_iteration
```

- Write the methods MStepViterbi( ), filling the method TrainViterbi defined below and provided in hmm.py.

```
def MstepViterbi(self,sum_B_num, sum_den, sum_A_num):
```

Question 4: Recognition

- Use the given program to train the 10 digit models. At each iteration of the EM algorithm the model is tested on the validation dataset by calling the `Log_likelihood(valid_data)`. The best model is saved through the iterations. The number of EM iterations is fixed to `Max_EM_iter`

- Comment and analyze the training and validation loss through the epochs, compare the curves with those of the froward Backward training.

- Run the recognition step on the MNIST test dataset composed of 10 000 digit images by putting `TRAINING = False`

- Fill in the table below and draw some conclusion by comparing these results with those obtained using Forward Backward training.

N_train	*n_cluster*	*n_states*	*RR (%) test*
10 000	*50*	*5*	-
	50	*10*	-
	50	*15*	-
	100	*5*	-
	100	*10*	-
10000	*100*	*15*	-
54 000	*100*	*15*	-